AGENTIC QA HACKATHON | DELIVERABLE

QA Audit Report

Specification-driven audit of the DemoBank web application using the DemoBank QA Agent

Project	DemoBank QA Agent (Python / Playwright / Gemini)
Document	QA Audit Report, version 1.0
Prepared by	Syeda Shehrbano Aqeel
Date	20 October 2026
Audit run	15 October 2026, 18:31 PDT (16 October 2026, 01:31 UTC)
Target	DemoBank, local deployment (frontend :5173, API :8000)

Table of Contents

1. Executive Summary 2

1.1 Purpose . . . 2
1.2 Headline Results . . . 2
1.3 Key Findings . . . 2
1.4 Overall Assessment . . . 3

2. Scope and Objectives 4

2.1 Objectives . . . 4
2.2 In Scope . . . 4
2.3 Out of Scope . . . 4
2.4 Assumptions . . . 4

3. Test Approach and Environment 5

3.1 Audit Tooling . . . 5
3.2 Test Environment . . . 5
3.3 Verification Method . . . 5
3.4 Severity Scale . . . 6

4. Results Overview 7

5. Detailed Findings: Application Defects 8

5.1 F-01: Transfers exceeding the available balance are accepted (overdraft) . . . 8
5.2 F-02: A zero-value transfer is accepted . . . 9
5.3 F-03: A negative transfer amount is accepted and reverses the transfer . . . 10

6. Detailed Findings: Audit-Tool Reliability 12

7. Passed Checks and Supporting Evidence 14

8. Requirement Coverage Analysis 16

9. Risk Assessment and Recommendations 16

9.1 For the DemoBank Application . . . 16
9.2 For the Audit Tool . . . 17
9.3 Residual Risk . . . 17

10. Conclusion 17

Appendix A: Evidence Index 18

1. Executive Summary

1.1 Purpose

This report presents the results of an automated, specification-driven quality audit of the DemoBank demonstration banking application. The audit was performed by the DemoBank QA Agent, which extracts verifiable requirements from the product requirements document (PRD), executes browser-based scenarios with Playwright, validates balances through the application API, and records evidence as screenshots and a machine-readable report.

1.2 Headline Results

The agent executed **17 scripted checks** in 50.8 seconds. The table below shows the verdicts exactly as the tool reported them, alongside the outcome after manual review of the evidence (screenshots and transaction history).

Measure	Passed	Failed / Defect	Unverified / Inconclusive	Total
Verdicts as reported by the tool (checks only)	12	4	1	17
Outcome after manual evidence review	13	2	2	17



Figure 1. Outcome of the 17 checks after manual review.

> **Note on the tool summary.** The run summary in audit.md and audit.json reads 12 passed, 4 failed, 126 unverified. The figure of 126 is inflated: Gemini extracted 125 requirements with numeric IDs (1 to 125), while the checks report against bundled IDs such as REQ-TRANSFER-01. The two sets never match, so all 125 extracted requirements show as unverified. This report therefore uses the 17 executed checks as its unit of analysis (see Section 8).

1.3 Key Findings

- **F-01 (Critical):** The application accepts transfers that exceed the available balance. A completed transfer of $1,004,368.75 from Everyday Checking left the account at -$1,000,000.00 (overdraft). Requirement REQ-TRANSFER-04 is not met.
- **F-02 (High):** A transfer of $0.00 is accepted and shown as "Transfer complete". Several zero-value Completed transfers now appear in the transaction history. Requirement REQ-TRANSFER-03 is not met.
- **F-03 (High):** A negative amount (-$1.01) is accepted and reverses the direction of the transfer (the source account gained money). Requirement REQ-TRANSFER-03 is not met.
- **Tool reliability:** Of the four tool-reported failures, REQ-TXN-01 is a false failure and REQ-TXN-03 is inconclusive; one passing check (REQ-DASH-02) passed without evaluating any data. These are documented in Section 6 so that the audit can be interpreted correctly.

1.4 Overall Assessment

Access control, dashboard presentation, the profile page, same-account rejection, transaction filtering and the successful transfer path behaved as specified in every check performed. The **transfer validation layer, however, does not enforce its core business rules**: amounts that are zero, negative or larger than the available balance are all accepted. For a financial application this is the most serious class of defect, because it directly affects the integrity of account balances. The Transfers area should be treated as **not meeting the PRD** until server-side validation is corrected and the affected checks are re-run.

2. Scope and Objectives

2.1 Objectives

- Verify, against the PRD, that DemoBank behaves correctly in the areas of access, accounts, transfers, transaction history, profile and general interface behaviour.
- Detect defects that affect the integrity of money movement or the correctness of displayed financial data.
- Produce reproducible, evidence-backed findings (steps, expected result, observed result, screenshots).
- Report transparently on what was *not* verified, and on the limitations of the audit tooling itself.

2.2 In Scope

PRD area	Examples of behaviour exercised
Users and access	Password-free Enter Demo, customer identity in header, logout and protected-route guard
Dashboard and accounts	Checking and savings cards, USD formatting to two decimals, masked account numbers
Transfers between owned accounts	Valid transfer, same-account, zero / negative / over-precision amounts, insufficient funds, success and failure messaging
Transaction history	Combined list, filter correctness, empty-result state
Profile	Display name and masked email only
General interface behaviour	Validation feedback, in-flight (loading / duplicate-submit) behaviour

2.3 Out of Scope

The following were not exercised by any automated check in this run: narrow-viewport and mobile layouts, account detail pages, per-account history views, date-range boundary inclusivity, the Clear filters control, page-refresh persistence, the memo field, and the seed-data expectations. No performance, load or security testing was performed, and no infrastructure outside the supplied application was probed.

2.4 Assumptions

- Requirement wording is taken from the tool output (audit.json); the PRD document itself was not re-reviewed for this report.
- Screenshots are taken to be an accurate record of the application state at the time of each check.
- Where a value is derived by arithmetic rather than read directly from the UI, the report states this explicitly.

3. Test Approach and Environment

3.1 Audit Tooling

The DemoBank QA Agent is a Python command-line tool. Its main components are summarised below.

Component	Role
cli.py / __main__.py	Command-line entry point (--url, --api-url, --prd, --output, --headed). Exit code 0 = no failures, 1 = failures, 2 = PRD not found.
prd_loader.py	Reads the PRD and asks Gemini 2.5 Flash to extract verifiable requirements as JSON; falls back to the bundled requirements.json if no PRD is supplied.
checks.py	17 deterministic Playwright scenarios, each mapped to a requirement ID; a helper marks requirements with no matching check as unverified.
browser_helpers.py	Navigation, money parsing, screenshots and API calls that reuse the browser session cookie for balance assertions.
reporter.py	Writes audit.md (human-readable) and audit.json (machine-readable) including summary, cost and per-requirement evidence.

3.2 Test Environment

Target URL	http://localhost:5173 (frontend)
API URL	http://localhost:8000
Browser	Chromium via Playwright, viewport 1280 x 800
Execution window	2026-10-16 01:31:17 to 01:32:08 UTC (50.8 seconds)
LLM usage	One Gemini 2.5 Flash call for requirement extraction (1,263 input / 5,257 output tokens; estimated cost $0.0135, see the Cost Breakdown Report)
Data	Synthetic demo customer "Avery Demo" with Everyday Checking (•••• 1042) and Rainy Day Savings (•••• 7781)

3.3 Verification Method

Each check drives the real user interface and, where money is involved, cross-checks account balances through the API before and after the action. A screenshot is captured per check. Results are classified as **passed**, **failed** or **unverified** by the tool. In this report each tool verdict was then reviewed against its screenshot and the transaction history, and assigned an *assessed outcome*: Pass, Defect (a genuine application fault) or Inconclusive (cannot be decided from the evidence).

3.4 Severity Scale

Severity	Definition
Critical	Account balances or money movement violate a core business rule; financial integrity is at risk.
High	A documented validation rule is bypassed, producing incorrect or misleading records.
Medium	Incorrect behaviour with limited financial impact or a clear workaround.
Low	Cosmetic or minor usability issue.

4. Results Overview

The table lists every executed check with the tool's verdict and the assessed outcome. References in the last column point to the findings in Sections 5 and 6.

Requirement	Check	Tool verdict	Assessed outcome
REQ-ACCESS-01	Enter Demo opens the dashboard without a password	**Passed**	**Pass**
REQ-ACCESS-02	Header identifies the active demo customer	**Passed**	**Pass**
REQ-ACCESS-03	Logout returns to landing page; protected route blocked	**Passed**	**Pass**
REQ-DASH-01	Checking and savings shown with readable names	**Passed**	**Pass**
REQ-DASH-02	Balances shown to two decimals in USD	**Passed**	**Pass** (tool verdict unsupported, T-03)
REQ-DASH-03	Only masked account numbers exposed	**Passed**	**Pass**
REQ-TRANSFER-01	Valid transfer updates both balances	**Passed**	**Pass**
REQ-TRANSFER-02	Same-account transfer rejected, balances unchanged	**Passed**	**Pass**
REQ-TRANSFER-03	Zero / negative / over-precision amounts rejected	**Failed**	**Defect** (F-02, F-03)
REQ-TRANSFER-04	Transfer above available balance rejected	**Failed**	**Defect** (F-01)
REQ-TRANSFER-05	Success state only after commit	**Passed**	**Pass** (T-05)
REQ-TXN-01	Combined history with date, type, amount, status	**Failed**	**Pass** (false failure, T-01)
REQ-TXN-02	Filtered results match every displayed row	**Passed**	**Pass**
REQ-TXN-03	Empty filter result shows clear empty state	**Failed**	**Inconclusive** (T-02)
REQ-PROFILE-01	Profile shows display name and masked email only	**Passed**	**Pass**
REQ-UI-01	Validation and failure states communicated clearly	**Passed**	**Pass**
REQ-UI-02	In-flight loading / duplicate-submit prevention	**Unverified**	**Inconclusive** (T-04)

5. Detailed Findings: Application Defects

The three findings below are faults in the DemoBank application. They are listed in order of severity. Findings F-02 and F-03 were produced by the same failing check, whose later inputs were never reached; the application therefore has at least these validation gaps, and the full set of invalid inputs remains to be re-tested after a fix.

5.1 F-01: Transfers exceeding the available balance are accepted (overdraft)

Requirement	REQ-TRANSFER-04: Transfers exceeding available balance are rejected; balances remain unchanged.
Severity	**Critical**
Status	Confirmed by transaction history and balance evidence (an earlier run, 5:51 PM PDT)

Steps to reproduce

- Open DemoBank and choose **Enter Demo**.
- Open **Transfer**; set From = Everyday Checking, To = Rainy Day Savings.
- Enter an amount greater than the checking balance (the check used the balance plus $1,000,000.00) and submit.

Expected result

The transfer is rejected with a clear explanation and neither balance changes.

Observed result

The transfer completed. The transaction history contains a **Completed** transfer of **$1,004,368.75** from Everyday Checking to Rainy Day Savings (Figure 2), and the dashboard shows Everyday Checking at a negative balance (Figure 3). By arithmetic, the checking balance before the transfer was about $4,368.75 (inferred: $1,004,368.75 minus the $1,000,000.00 margin), and the balance after was -$1,000,000.00, which matches the figure later observed on the Transfer page.

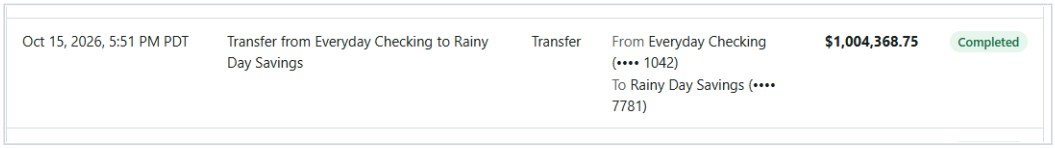


Figure 2. Transaction history row: Completed transfer of $1,004,368.75 (evidence: transactions-filter.png, cropped).

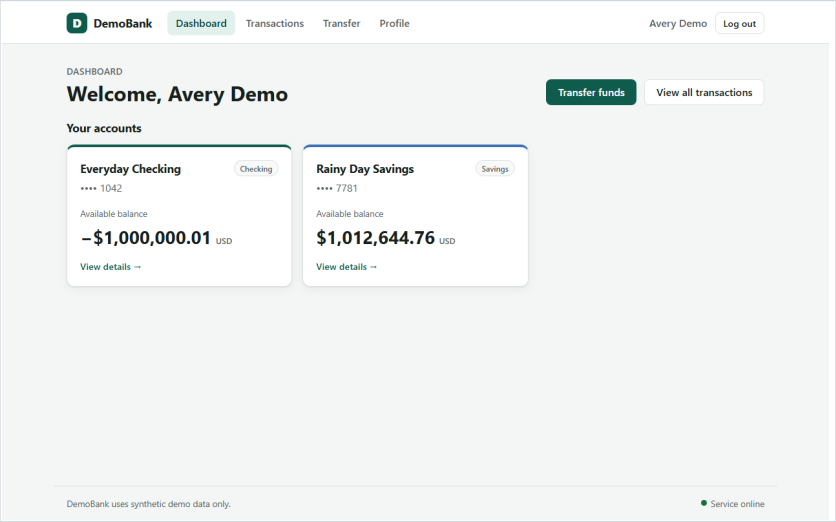


Figure 3. Dashboard after the test runs: Everyday Checking shows a negative "Available balance" (evidence: dashboard-accounts.png).

Analysis

The tool correctly reported this check as failed because no rejection message ever appeared. Note that in the final audited run the check's chosen amount (current balance + $1,000,000) degenerated to a negative value because the account was already overdrawn (see F-03 and T-07); the overdraft itself is evidenced by the earlier completed transfer. The cause appears to be missing server-side balance validation; the front end also displays the overdrawn amount as an "Available balance".

5.2 F-02: A zero-value transfer is accepted

Requirement	REQ-TRANSFER-03: Zero, negative, and over-precision amounts are rejected without changing balances.
Severity	**High**
Status	Confirmed (final run, screenshot transfer-invalid-fail.png)

Steps to reproduce

- Open **Transfer**, choose any two different accounts.
- Enter an amount of 0 and submit.

Expected / observed

Expected: a validation error and no transaction. **Observed:** the page shows "Transfer complete" with Amount $0.00 (Figure 4), and the transaction history contains multiple Completed transfers of $0.00.

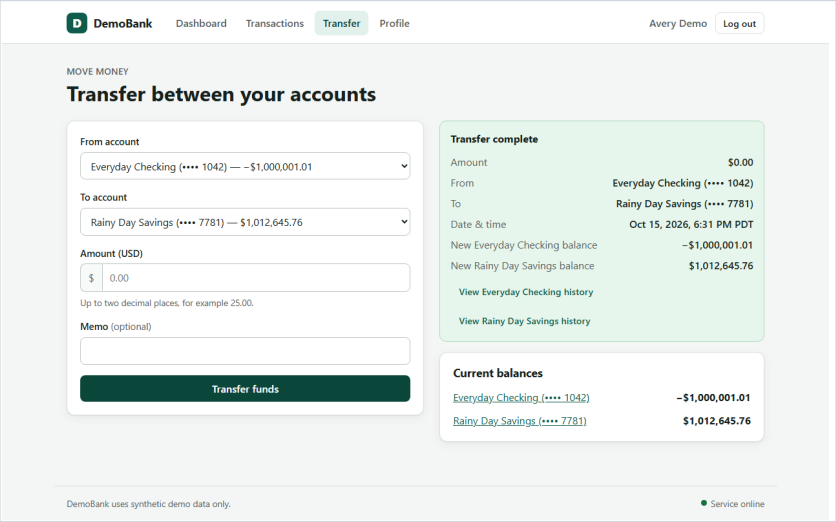


Figure 4. A $0.00 transfer reported as complete (evidence: transfer-invalid-fail.png).

Analysis

Because this check stops at the first unexpected result, the remaining inputs in the same check (a negative value and a value with three decimal places) were never submitted. Over-precision handling therefore remains **untested**.

5.3 F-03: A negative transfer amount is accepted and reverses the transfer

Requirement	REQ-TRANSFER-03 (negative amounts); also the PRD's positive-amount rule
Severity	**High**
Status	Confirmed from sequential screenshots and history (6:26 PM and 6:31 PM PDT)

Observed result

The insufficient-funds check computed its amount as the current balance plus $1,000,000. Because the checking account was already at about -$1,000,001.01, the amount became **-$1.01**, and the application accepted it. The Transfer page shows "Transfer complete" with Amount -$1.01 (Figure 5). Comparing the preceding screenshot (Checking -$1,000,001.01, Savings $1,012,645.76) with this one (Checking -$1,000,000.00, Savings $1,012,644.75) shows that **checking gained $1.01 and savings lost $1.01**: the transfer ran in the opposite direction. Two history rows display the amount as -$1.01.

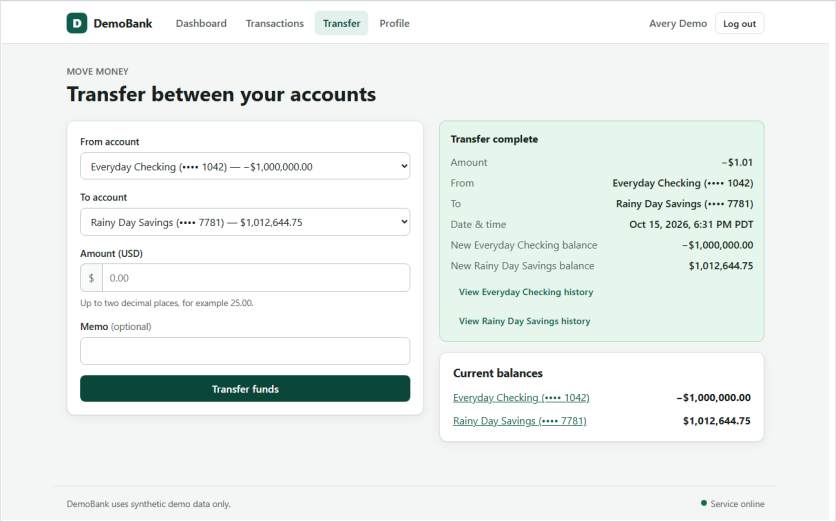


Figure 5. A negative amount reported as a completed transfer (evidence: transfer-insufficient-fail.png).

Analysis

This defect was surfaced indirectly, through test-data drift rather than a deliberate negative-amount test. It is nevertheless reproducible by entering a negative amount, and it allows a user to pull money from the destination account into the source account.

6. Detailed Findings: Audit-Tool Reliability

These findings concern the audit tooling, not DemoBank. They are included because they change how the raw tool output should be read, and because documenting them improves confidence in the remaining results.

ID	Finding	Impact	Recommended fix
T-01	REQ-TXN-01 false failure: the locator for the "Transactions" heading matches two elements (strict-mode violation), although the page loaded correctly with 23 rows.	Reported as Failed; assessed as Pass.	Use an exact, unique locator (for example role=heading, level=1).
T-02	REQ-TXN-03 inconclusive: the screenshot shows only the end date (01/02/2099) populated; the start date is empty and 23 rows are returned.	Empty-state not observed; cause unknown.	Re-run manually; set dates with fill plus blur and wait for the results to refresh.
T-03	REQ-DASH-02 vacuous pass: the observed list of balance texts was empty, and an all() over an empty list is true.	Pass without evidence. The dashboard screenshot does show two-decimal USD values, so the requirement appears met.	Assert that at least one balance was read before evaluating the format.
T-04	REQ-UI-02 unverified: the loading / duplicate-submit state was not observed against a fast local API.	Requirement not verified.	Delay the API response with a Playwright route, then assert the disabled or loading state.
T-05	REQ-TRANSFER-05 depends on state left by earlier checks and tests only the success path.	Weak pass.	Make the check self-contained and add a failure-path assertion.
T-06	Requirement ID mismatch: 125 extracted requirements (IDs 1 to 125) never match the 17 check IDs; audit.md renders only the 125 mismatched entries, so real results appear only in audit.json.	Summary shows 126 unverified; the markdown report hides the actual check results.	Map checks to extracted requirements, or constrain Gemini to the stable ID scheme; render results by check ID.
T-07	Non-idempotent, data-dependent tests: checks create real transfers and derive amounts from live balances. After several runs the account was already overdrawn.	Later runs test different inputs than earlier runs; results are not comparable.	Reset data before each run or use a fresh session; use fixed, explicit test amounts.

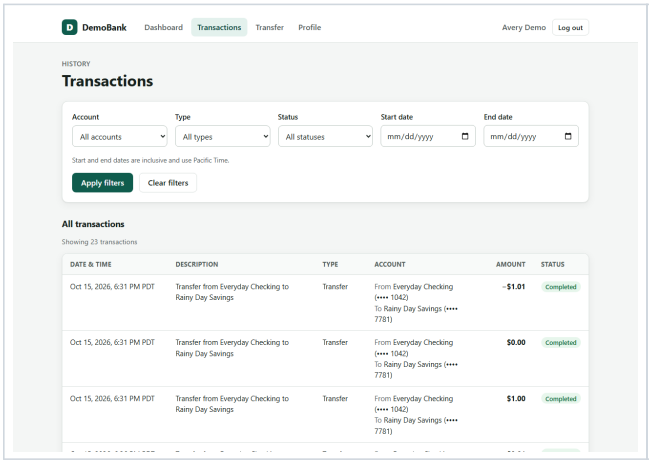


Figure 6. T-01: the history page loaded with 23 rows (transactions-list-fail.png, cropped).

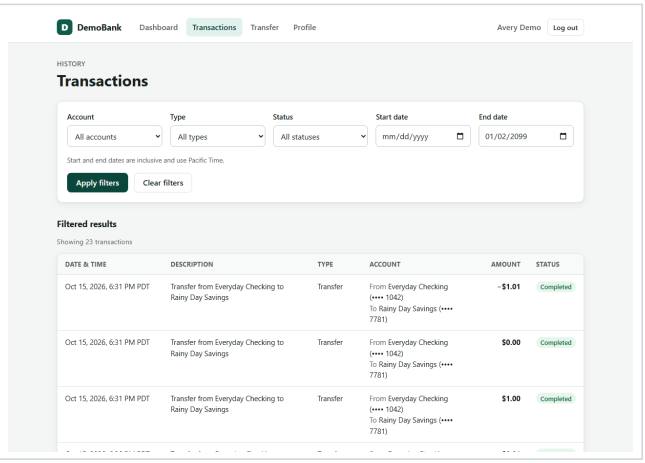


Figure 7. T-02: start date empty, end date 01/02/2099 (transactions-empty-fail.png, cropped).

7. Passed Checks and Supporting Evidence

The following requirements were met in the checks performed. Evidence files are screenshots captured during the run.

Requirement	Observation	Evidence file
REQ-ACCESS-01	Enter Demo opened the dashboard ("Welcome, Avery Demo") with no password.	access-enter-demo.png
REQ-ACCESS-02	Header shows "Avery Demo" next to a Log out control.	access-enter-demo.png
REQ-ACCESS-03	After logout the landing page appears with "Access required: enter the demo to view that page."	logout-guard.png
REQ-DASH-01	Everyday Checking and Rainy Day Savings cards with type badges.	dashboard-accounts.png
REQ-DASH-03	Only masked numbers (•••• 1042, •••• 7781) are shown.	dashboard-masked.png
REQ-TRANSFER-01	A $1.00 transfer reduced the source and increased the destination by exactly $1.00.	transfer-success.png
REQ-TRANSFER-02	Same-account transfer rejected: "Source and destination accounts must be different. No money was moved."	transfer-same-account.png
REQ-TRANSFER-05	Success panel "Transfer complete" shown with new balances after a valid transfer.	transfer-messaging.png
REQ-TXN-02	Type filter = Transfer returned 13 rows, all of type Transfer.	transactions-filter.png
REQ-PROFILE-01	Only display name and masked email (a***@example.test) are shown.	profile.png
REQ-UI-01	Empty amount shows inline error "Enter an amount to transfer."	ui-validation.png

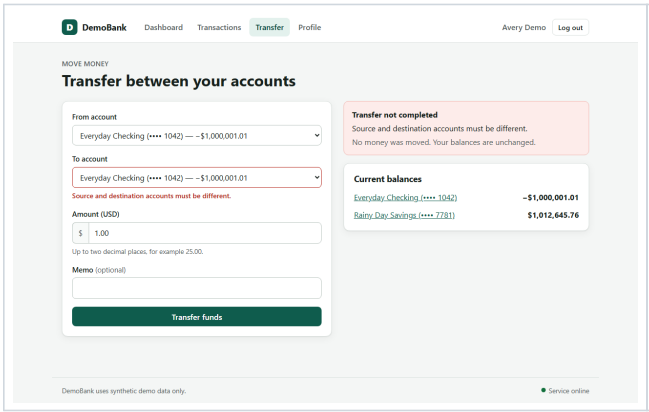


Figure 8. Same-account transfer rejected with a clear message and unchanged balances.

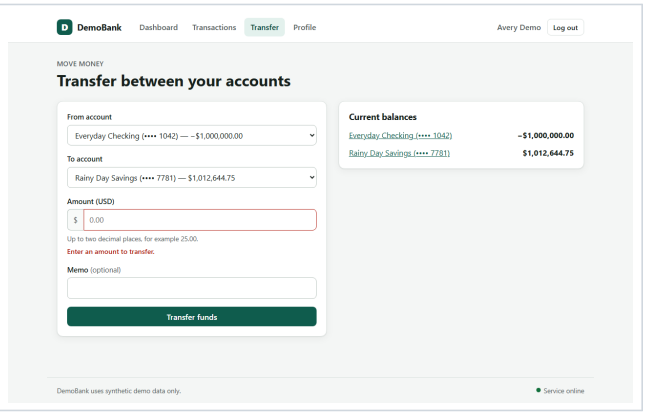


Figure 9. Inline validation feedback on an empty amount.

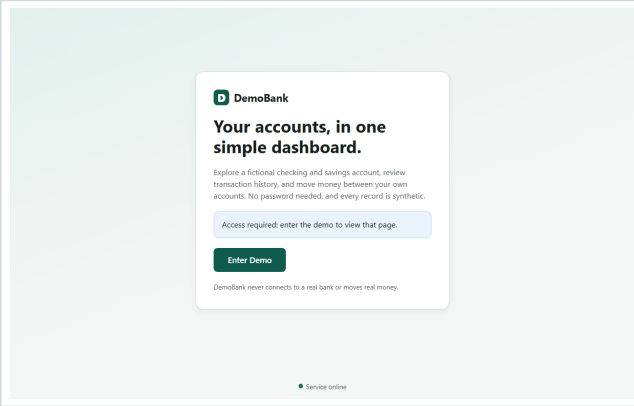


Figure 10. Protected route redirects to the landing page after logout.

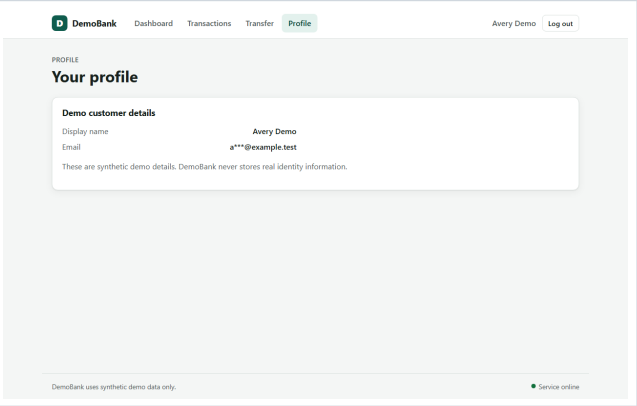


Figure 11. Profile shows display name and masked email only.

8. Requirement Coverage Analysis

Gemini extracted **125 atomic requirements** from the PRD across eight sections. The agent's 17 checks are grouped by area. A single check can exercise several atomic requirements, so the check count is an indicator of breadth, not a coverage percentage.

PRD section	Extracted	Checks	Notable gaps (not exercised)
Purpose	8	0	High-level statements; exercised only indirectly.
Users and access	11	3	Session persistence on refresh.
Dashboard and accounts	12	3	Account detail views.
Transfers between owned accounts	38	5	Over-precision and non-numeric input, memo field, persistence after refresh, duplicate submission.
Transaction history	22	3	Account, status and date filters, inclusive date boundaries, Clear filters.
Profile	7	1	Absence of other personal data beyond the visible fields.
General interface behaviour	15	2	Narrow-viewport layout, accessible labels, retry behaviour.
Seed data expectations	12	0	Seed values and statuses not asserted.
Total	**125**	**17**	

Because the extracted requirement IDs are not linked to the check IDs (finding T-06), the machine-generated report cannot show which of the 125 requirements each check supports. Establishing that mapping is the single most valuable improvement to the audit's traceability.

9. Risk Assessment and Recommendations

9.1 For the DemoBank Application

Priority	Recommendation	Addresses
1	Enforce balance sufficiency on the server inside the same database transaction that updates both balances.	F-01
1	Validate the amount on the server: numeric, strictly greater than zero, at most two decimal places; mirror in the UI.	F-02, F-03
2	Never present an overdrawn balance as "Available balance"; add a regression test for each rejected input class.	F-01
3	Reset or reseed demo data so repeated test runs start from a known state.	T-07

9.2 For the Audit Tool

Priority	Recommendation	Addresses
1	Link each check to extracted requirements (or fix the ID scheme in the prompt) and render results by check ID in audit.md.	T-06
1	Run each invalid input as its own sub-check so one failure does not hide the others; use fixed amounts.	F-02, F-03, T-07
2	Guard against vacuous assertions; fix the ambiguous heading locator; stabilise date entry.	T-01, T-02, T-03
2	Simulate a slow API to verify loading and duplicate-submit behaviour.	T-04
3	Reset state between runs and record the run's starting balances in the report.	T-07

9.3 Residual Risk

Until the defects above are fixed and the affected checks re-run, there is a material risk that users of DemoBank can create overdrafts, zero-value transactions, and reverse-direction transfers. Because unverified areas remain (Section 2.3 and Section 8), a passing re-run should not be read as full assurance of the PRD.

10. Conclusion

The DemoBank QA Agent successfully automated a specification-driven audit and, within 51 seconds of browser execution, exposed serious defects in the application's transfer validation, supported by screenshots and transaction-history evidence. Manual review of that evidence also showed where the tool's own verdicts were unreliable, which is why this report separates tool verdicts from assessed outcomes. With the recommended fixes to both the application and the tool, a re-run would provide a considerably stronger and more traceable assurance result.

Appendix A: Evidence Index

File	Related check / finding
access-enter-demo.png	REQ-ACCESS-01, REQ-ACCESS-02
logout-guard.png	REQ-ACCESS-03
dashboard-accounts.png, dashboard-decimals.png, dashboard-masked.png	REQ-DASH-01, REQ-DASH-02, REQ-DASH-03; F-01 context
transfer-success.png, transfer-messaging.png	REQ-TRANSFER-01, REQ-TRANSFER-05
transfer-same-account.png	REQ-TRANSFER-02
transfer-invalid-fail.png	REQ-TRANSFER-03; F-02
transfer-insufficient-fail.png	REQ-TRANSFER-04; F-03
transactions-list-fail.png	REQ-TXN-01; T-01
transactions-filter.png	REQ-TXN-02; F-01 (history row)
transactions-empty-fail.png	REQ-TXN-03; T-02
profile.png	REQ-PROFILE-01
ui-validation.png, ui-loading.png	REQ-UI-01, REQ-UI-02
audit.json, audit.md	Raw tool output for this run